

# **TOPIC 05**

# **ELEMENTARY PROGRAMMING**

## **(PART 2)**

Fundamentals of Computing  
(FSPK0022)

# Arithmetic Expression

# Arithmetic Operators and Expression

# Arithmetic Operators

- Used for performing numeric calculations
- C++ has unary, binary, and ternary operators
  - unary (1 operand)       $-5$
  - binary (2 operands)     $13 - 7$
  - ternary (3 operands)    $\text{exp1} ? \text{exp2} : \text{exp3}$

# Binary Arithmetic Operators

| Symbol | Operation      | Example                   | ans |
|--------|----------------|---------------------------|-----|
| +      | Addition       | <code>ans = 7 + 3;</code> | 10  |
| -      | Subtraction    | <code>ans = 7 - 3;</code> | 4   |
| *      | Multiplication | <code>ans = 7 * 3;</code> | 21  |
| /      | Division       | <code>ans = 7 / 3;</code> | 2   |
| %      | Modulus        | <code>ans = 7 % 3;</code> | 1   |

# / Operator

- C++ division operator (/) performs integer division if both operands are integers

```
cout << 13 / 5;    // displays 2  
cout << 2 / 4;    // displays 0
```

- If either operand is floating-point, the result is floating-point

```
cout << 13 / 5.0;  // displays 2.6  
cout << 2.0 / 4;   // displays 0.5
```

# % Operator

- C++ modulus operator (%) computes the remainder resulting from integer division

```
cout << 9 % 2;    // displays 1
```

- % requires integers for both operands

```
cout << 9 % 2.0;  // error
```

# In-Class Exercise

- Identify as many syntax errors as you can in the following program:

```
*/ what is wrong with this program?/*  
#include iostream  
using namespace std;  
  
int main();  
{  
    int a, b, c  
    a=3  
    b=4  
    c=a+b  
    Cout<<"The value of c is "<<C;  
    return 0;  
}
```



# Order of Operations

In an expression with more than one operator, evaluation is in this order:

( )

– (unary negation), in order, right to left

\* / %, in order, left to right

+ –, in order, left to right

In the expression  $2 + 2 * 2 - 2$



evaluate  
second



evaluate  
first



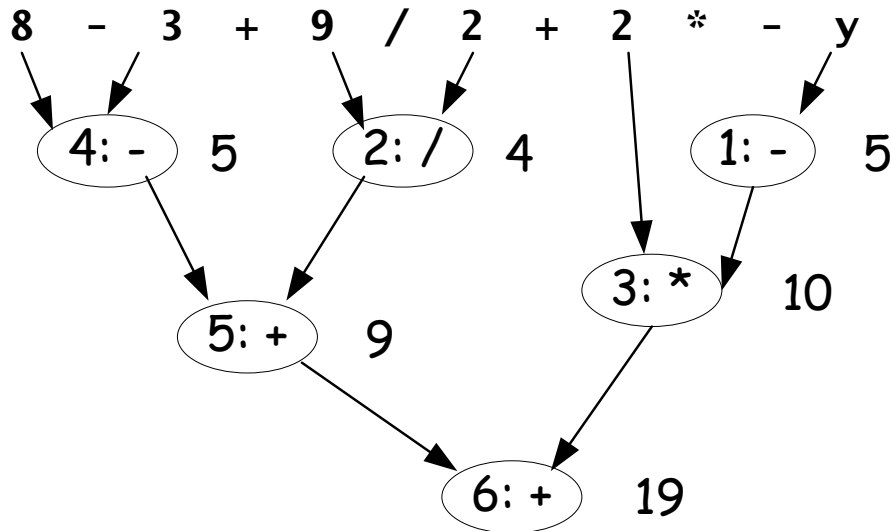
evaluate  
third

# Example

```
int z, y=-5;
```

```
z= 8 - 3 + 9 / 2 + 2 * - y;
```

```
z= 8 - (3 + 9 / 2) + 2 * - y; // try this
```



# Order of Operations

Show prove for the following expression:

**Table 3-2 Some Expressions**

| Expression          | Value |
|---------------------|-------|
| $5 + 2 * 4$         | 13    |
| $10 / 2 - 3$        | 2     |
| $8 + 12 * 2 - 4$    | 28    |
| $4 + 17 \% 2 - 1$   | 4     |
| $6 - 3 * 2 + 7 - 1$ | 6     |

# Associativity of Operators

- $-$  (unary negation) associates right to left
- $*$ ,  $/$ ,  $\%$ ,  $+$ ,  $-$  associate left to right
- parentheses  $( )$  can be used to override the order of operations:

$$2 + 2 * 2 - 2 = 4$$

$$(2 + 2) * 2 - 2 = 6$$

$$2 + 2 * (2 - 2) = 2$$

$$(2 + 2) * (2 - 2) = 0$$

# Grouping with Parentheses

**Table 3-4 More Expressions**

| Expression              | Value |
|-------------------------|-------|
| $(5 + 2) * 4$           | 28    |
| $10 / (5 - 3)$          | 5     |
| $8 + 12 * (6 - 2)$      | 56    |
| $(4 + 17) \% 2 - 1$     | 0     |
| $(6 - 3) * (2 + 7) / 3$ | 9     |

# Type Conversion

# When You Mix Apples and Oranges: *Type Conversion*

- Operations are performed between operands of the same type.
- If not of the same type, C++ will convert one to be the type of the other
- This can impact the results of calculations.

# Type Conversion

- Type Conversion: automatic conversion of an operand to another data type
- Promotion: convert to a higher type
- Demotion: convert to a lower type



# Hierarchy of Types

Highest:   long double  
              double  
              float  
              unsigned long  
              long  
              unsigned int  
              int

Lowest:

Ranked by largest number they can hold

# Conversion Rules

- 1) `char`, `short`, `unsigned short` automatically promoted to `int`

- For arithmetic operation:

```
char c='A'; cout << 6+c; //int
```

- 2) When operating on values of different data types, the lower one is promoted to the type of the higher one.

```
int i = 25; cout << 6.1 + i; //float
```

- 3) When using the `=` operator, the type of expression on right will be converted to type of variable on left

```
int x, y = 25; float z = 2.5;
```

```
x = y + z; //int
```

# Overflow and Underflow

# Overflow and Underflow

- Occurs when assigning a value that is too large (overflow) or too small (underflow) to be held in a variable
- Variable contains value that is 'wrapped around' set of possible values
- Different systems may display a warning/ error message, stop the program, or continue execution using the incorrect value

# Type Casting

# Type Casting

- Used for manual data type conversion
- Useful for floating point division using int:

```
double m;  
m = static_cast<double>(y2-y1) / (x2-x1) ;
```

- Useful to see **int** value of a **char** variable:

```
char ch = 'C' ;  
cout << ch << " is "  
      << static_cast<int>(ch) ;
```

# Example

## Program 3-10

```
1  // This program uses a type cast to avoid integer division.
2  #include <iostream>
3  using namespace std;
4
5  int main()
6  {
7      int books;           // Number of books to read
8      int months;          // Number of months spent reading
9      double perMonth;     // Average number of books per month
10
11     cout << "How many books do you plan to read? ";
12     cin >> books;
13     cout << "How many months will it take you to read them? ";
14     cin >> months;
15     perMonth = static_cast<double>(books) / months;
16     cout << "That is " << perMonth << " books per month.\n";
17     return 0;
18 }
```

### Program Output with Example Input Shown in Bold

How many books do you plan to read? **30 [Enter]**

How many months will it take you to read them? **7 [Enter]**

That is 4.28571 books per month.

# C-Style and Prestandard Type Cast Expressions

- C-Style cast: data type name in ( )

```
cout << ch << " is " << (int)ch;
```

- Prestandard C++ cast: value in ( )

```
cout << ch << " is " << int(ch);
```

- Both are still supported in C++, although **static\_cast** is preferred



# Algebraic Expressions

# Algebraic Expressions

- Multiplication requires an operator:

$Area = lw$  is written as `Area = l * w;`

- There is no exponentiation operator:

$Area = s^2$  is written as `Area = pow(s, 2);`

- Parentheses may be needed to maintain order of operations:

$$m = \frac{y^2 - y^1}{x_2 - x_1}$$

is written as

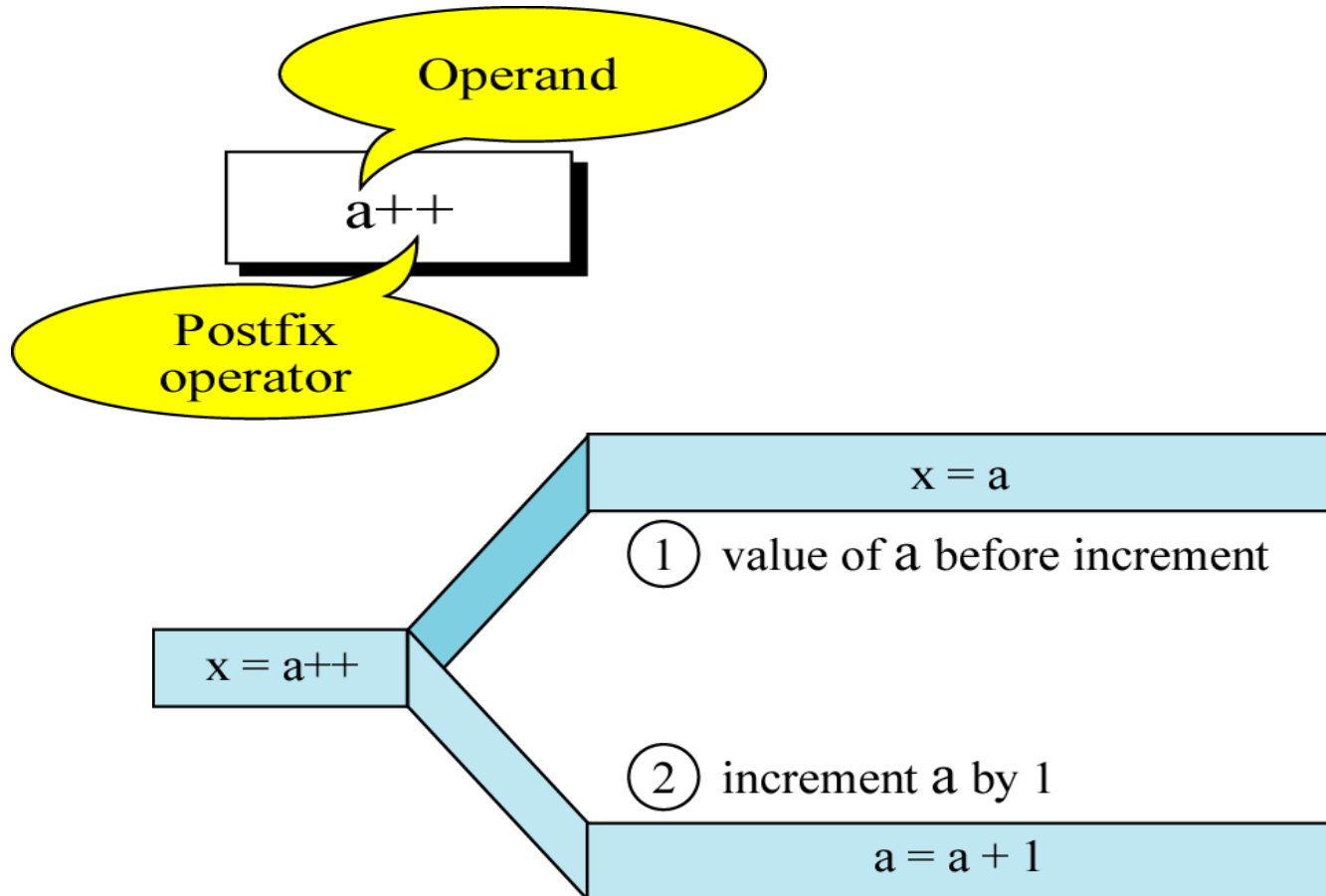
$$m = (y^2 - y^1) / (x_2 - x_1);$$

# Algebraic Expressions

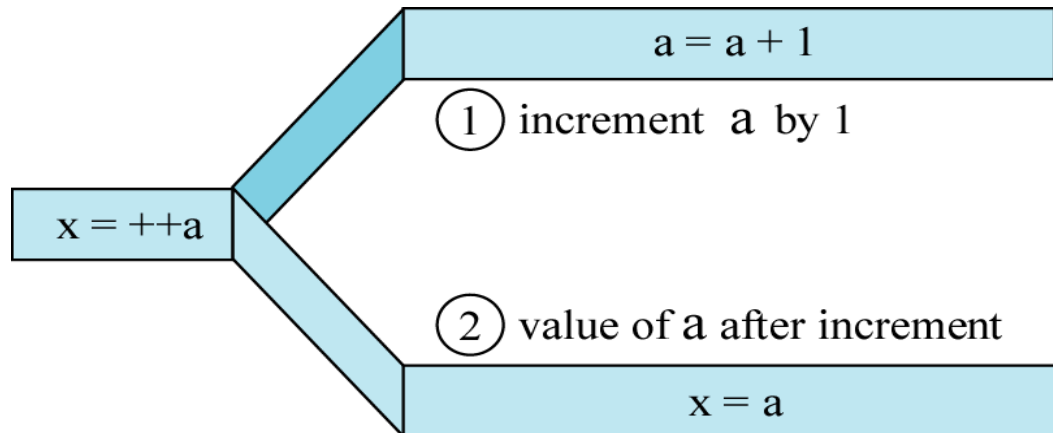
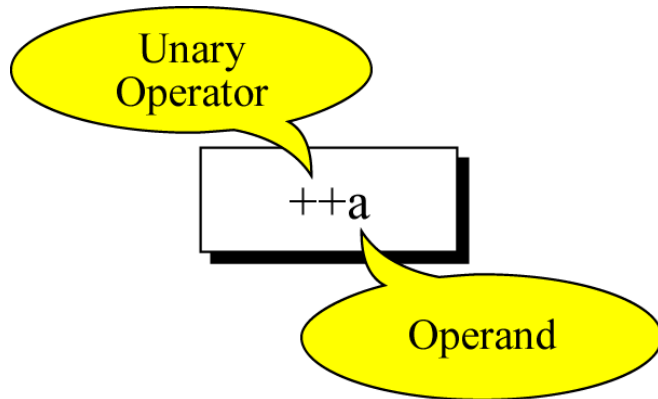
**Table 3-5 Algebraic and C++ Multiplication Expressions**

| Algebraic Expression | Operation         | C++ Equivalent         |
|----------------------|-------------------|------------------------|
| $6B$                 | 6 times B         | <code>6 * B</code>     |
| $(3)(12)$            | 3 times 12        | <code>3 * 12</code>    |
| $4xy$                | 4 times x times y | <code>4 * x * y</code> |

# Postfix expression



# Prefix expression



# In-Class Exercise

- What would be the value of `nilai_kedua`:

```
int kira = 5;
```

```
int nilai_pertama = 10, nilai_kedua;
```

```
nilai_kedua= 5* kira-- + nilai_pertama;
```

```
nilai_kedua = 5* --kira + nilai_pertama;
```

# Multiple Assignment and Combined Assignment

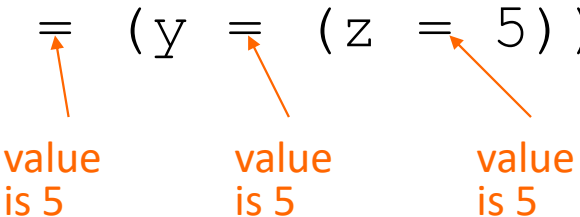
# Multiple Assignment and Combined Assignment

- The = can be used to assign a value to multiple variables:

`x = y = z = 5;`

- Value of = is the value that is assigned
- Associates right to left:

`x = (y = (z = 5)) ;`



value is 5      value is 5      value is 5



# Combined Assignment

- Look at the following statement:

```
sum = sum + 1;
```

This adds 1 to the variable **sum**.

# Combined Assignment

- The combined assignment operators provide a shorthand for these types of statements.
- The statement

```
sum = sum + 1;
```

is equivalent to

```
sum += 1;
```

# Combined Assignment Operators

| Operator  | Example                      | Equivalent to                      |
|-----------|------------------------------|------------------------------------|
| <b>+=</b> | $i += 3$<br>$i += j + 3$     | $i = i + 3$<br>$i = i + (j + 3)$   |
| <b>-=</b> | $i -= 3$<br>$i -= j + 3$     | $i = i - 3$<br>$i = i - (j + 3)$   |
| <b>*=</b> | $i *= 3$<br>$i *= j + 3$     | $i = i * 3$<br>$i = i * (j + 3)$   |
| <b>/=</b> | $i /= 3$<br>$i /= j + 3$     | $i = i / 3$<br>$i = i / (j + 3)$   |
| <b>%=</b> | $i \% = 3$<br>$i \% = j + 3$ | $i = i \% 3$<br>$i = i \% (j + 3)$ |

# In-Class Exercise

Assume that `int a = 1` and `double d = 1.0`, and that each expression is independent. What are the results of the following expressions?

- i) `a = 46/9;`
- ii) `a = 46 % 9 + 4 * 4 - 2;`
- iii) `a = 45 + 43 % 5 * (23 * 3 % 2);`
- iv) `a %= 3 / a + 3;`
- v) `d += 1.5 * 3 + (++a);`
- vi) `d -= 1.5 * 3 + a++;`

# Logical Expression

# Boolean and Logical Operator

- In C++ logical data declared as **bool** data type

e.g.

```
bool variable_name;
```

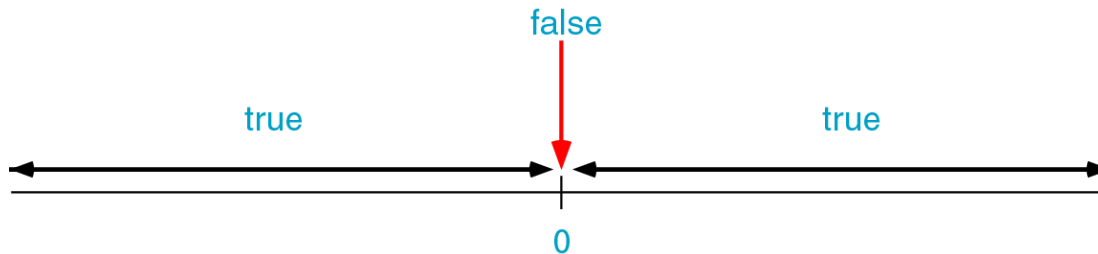
- There are only two values: **true** and **false**
- Type-casting **bool** to **int**:
  - **true** => 1
  - **false** => 0

*Example*

```
int number;  
number = 2 + true;  
cout << number; //output: 3
```

# Boolean and Logical Operator

- Type-casting `int` to `bool`:
  - A *Zero value* => `false`
  - A *Non-Zero value* => `true`



Example:

```
bool b = false;      // b initially is false
int number = 0;
b = -10;             // Now, b is true
b = number;          // Here, b is false again
```

# Boolean and Logical Operator

What would be printed by this code segment?

```
bool b;  
int p;  
int q = 5;  
  
b = q;  
p = b;  
cout << "The value of p is " << p << endl;
```

**Output:**

The value of p is 1



# Logical Operators Truth Table

not

| <b>x</b> | <b>!x</b> |
|----------|-----------|
| false    | true      |
| true     | false     |

logical

!

| <b>x</b> | <b>!x</b> |
|----------|-----------|
| zero     | 1         |
| nonzero  | 0         |

C Language

and

| <b>x</b> | <b>y</b> | <b>x&amp;&amp; y</b> |
|----------|----------|----------------------|
| false    | false    | false                |
| false    | true     | false                |
| true     | false    | false                |
| true     | true     | true                 |

logical

&&

| <b>x</b> | <b>y</b> | <b>x&amp;&amp; y</b> |
|----------|----------|----------------------|
| zero     | zero     | 0                    |
| zero     | nonzero  | 0                    |
| nonzero  | zero     | 0                    |
| nonzero  | nonzero  | 1                    |

C Language

or

| <b>x</b> | <b>y</b> | <b>x   y</b> |
|----------|----------|--------------|
| false    | false    | false        |
| false    | true     | true         |
| true     | false    | true         |
| true     | true     | true         |

logical

||

| <b>x</b> | <b>y</b> | <b>x    y</b> |
|----------|----------|---------------|
| zero     | zero     | 0             |
| zero     | nonzero  | 1             |
| nonzero  | zero     | 1             |
| nonzero  | nonzero  | 1             |

C Language

# Operations for Logical and/ or

false && (anything)



false

true || (anything)



true

# Relational Operators

| Operator | Meaning               |
|----------|-----------------------|
| <        | less than             |
| <=       | less than or equal    |
| >        | greater than          |
| >=       | greater than or equal |
| ==       | equal                 |
| !=       | not equal             |

# Logical Expression

## Example:

```
int a = 10;
```

```
cout << a;
```

Prints **10**

```
cout << (a == 1);
```

Prints **0**. Is  $10 == 1$  ?  $\Rightarrow$  false  $\Rightarrow$  0

```
cout << (a > 1);
```

Prints **1**. Is  $10 > 1$  ?  $\Rightarrow$  true  $\Rightarrow$  1

```
cout << (a = 5);
```

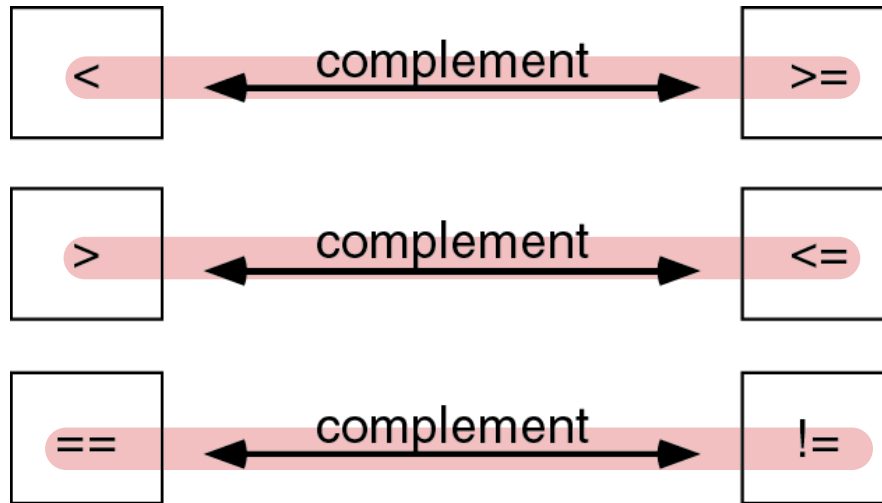
Prints **5**. This is not a logical expression. It is an assignment expression.

```
a = (a != 5);
```

```
out << a;
```

Prints **0**. Is  $5 != 5$  ?  $\Rightarrow$  false  $\Rightarrow$  0

# Logical Operator Complements



- Another way to complement an expression is just putting a **Not** operator (!) in front of it.

Example: Complement of `n==0` is **! (n==0)**

# In-Class Exercise

Determine the output for code segment below:

```
int x = 3, y = 5;
char code = 'A', code2 = 'S';
bool p = false;
bool q;

cout << ((x + 3) > (y + 5)) << endl;
cout << ((p != 0) + 10) && ((p + 10) == 10)) << endl;
cout << ((code == 'C') || (code2 != 'S')) << endl;

q = x - y;
x = q + 1;
cout << "The value of q is " << q << endl;
cout << "The value of x is " << x << endl;
```